

Relatório Técnico - Sistema de Chat Distribuído

- **Disciplina:** Sistemas Distribuídos
 - **Alunos:** Gustavo Rodrigues Ribeiro - RA:202003570 / Victor Fonseca Santana - RA:202004697 / Luiz Gustavo Pontes de Araújo - RA:202109901
-

1. Introdução

O desenvolvimento de sistemas de comunicação distribuídos representa um desafio fundamental na computação moderna. Este trabalho implementa um sistema de chat distribuído que simula funcionalidades básicas de aplicações como WhatsApp, utilizando sockets TCP para comunicação confiável entre múltiplos usuários.

1.1 Objetivos

- Implementar comunicação em tempo real entre usuários via sockets TCP ('Privada' ou 'Em Grupo')
- Desenvolver sistema de grupos com controle de acesso
- Permitir transferência de arquivos entre usuários e grupos
- Garantir concorrência segura para múltiplos usuários simultâneos
- Aplicar conceitos de sistemas distribuídos: sincronização, tratamento de falhas e protocolos de comunicação

1.2 Desafios Técnicos

Os principais desafios incluem sincronização de dados compartilhados entre threads, gerenciamento consistente de estado de usuários e grupos, tratamento de desconexões inesperadas, e definição de protocolo eficiente para comunicação cliente-servidor.

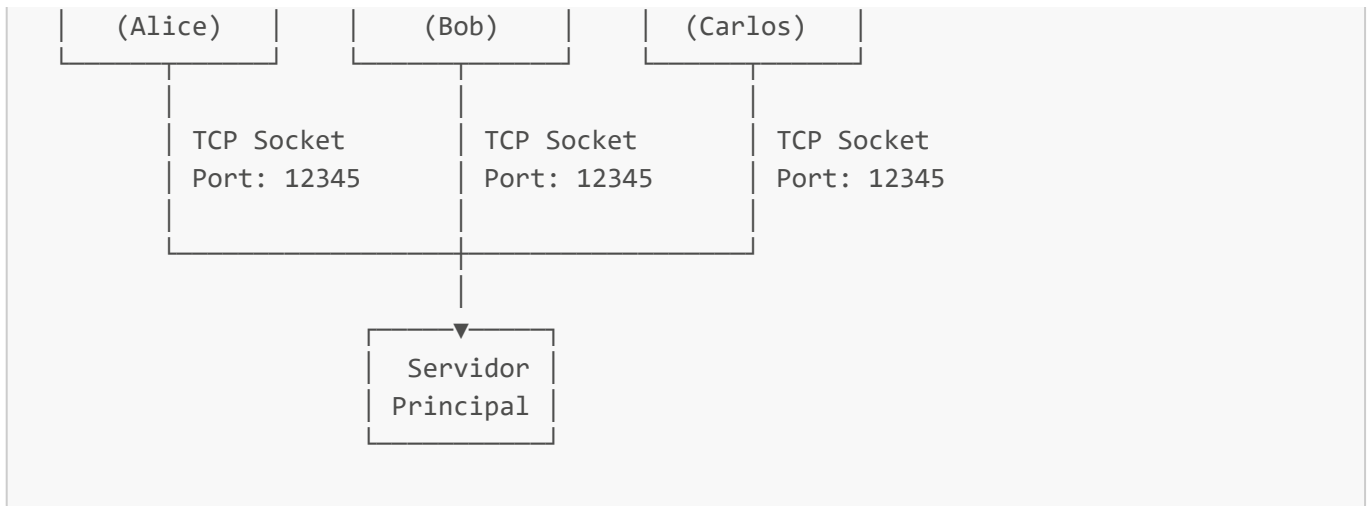
2. Arquitetura do Sistema

2.1 Visão Geral

O sistema adota arquitetura cliente-servidor centralizada com comunicação via sockets TCP. Um servidor único gerencia todas as conexões, mensagens e operações, oferecendo simplicidade de implementação e consistência de dados, embora apresente ponto único de falha. Os clientes estabelecem conexões TCP persistentes exclusivamente com este servidor.

Escolhemos essa arquitetura por sua simplicidade e eficácia em garantir a consistência dos dados. O servidor torna-se a fonte única de gerenciamento para o estado do sistema (usuários online, composição de grupos), eliminando a complexidade de algoritmos de consenso ou sincronização de estado que seriam necessários em modelos descentralizados (P2P). Embora introduza um ponto único de falha, para o escopo deste projeto, os benefícios em termos de simplicidade de desenvolvimento e controle superam essa limitação.





2.2 Componentes Principais

Servidor (ChatServer): O servidor é o núcleo da aplicação, orquestrando todas as operações. Suas responsabilidades incluem:

- Gerenciamento de conexões TCP simultâneas
- Roteamento de mensagens privadas e em grupo
- Controle de acesso e membership de grupos
- Armazenamento temporário de arquivos
- Coordenação thread-safe de operações

Cliente (ChatClient): O cliente é a interface do usuário, projetado para ser leve e responsivo, via terminal (CLI):

- Interface CLI para interação do usuário
- Thread dedicada para recepção assíncrona de mensagens
- Codificação/decodificação de arquivos em Base64
- Gerenciamento de conexão TCP com servidor

2.3 Estruturas de Dados Críticas

A seguir, temos as estruturas de dados utilizadas para gerenciar a quantidade e quais são os usuários (clientes) conectados utilizando o sistema:

```
# Gerenciamento de clientes conectados
self.clients: Dict[str, socket.socket] = {}
# Mapeamento: {"username": socket_object}

# Gerenciamento de grupos
self.groups: Dict[str, Set[str]] = {}
# Mapeamento: {"group_name": set_of_usernames}
```

2.4 Protocolo de Comunicação

Todas as mensagens utilizam protocolo de aplicação sobre TCP, utilizando JSON, com estrutura padronizada, para a serialização de dados:

```
{
  "type": "message_type", // Tipo de Mensagem
  "sender": "username", // Usuário que está enviando a mensagem
  "recipient": "target", // Usuário que irá receber a mensagem
  "content": "message_content", // Conteúdo da Mensagem
  "timestamp": "HH:MM:SS" // Horário de envio
}
```

Tipos de Mensagem:

- **login**: Autenticação inicial
- **private_message**: Comunicação direta entre usuários
- **group_message**: Mensagem para grupo
- **create_group**: Criação de novo grupo
- **add_member**: Adição de membro ao grupo
- **send_file**: Transferência de arquivo (Base64)

2.5 Fluxo de Comunicação

2.5.1 Fluxo de Mensagem Privada

 Fluxo de Mensagem Privada

2.5.2 Fluxo de Mensagem em Grupo

 Fluxo de Mensagem em Grupo

2.5.3 Fluxo de Adição de Membro ao Grupo

 Fluxo de Adição de Membro ao Grupo

3. Principais Decisões Técnicas

3.1 Escolha do Protocolo TCP

Justificativas:

A escolha do TCP (Transmission Control Protocol) foi para garantir a integridade da comunicação. TCP oferece:

- **Entrega garantida**: Essencial para aplicações de chat onde perda de mensagens é inaceitável.
- **Ordem preservada**: Mensagens chegam na sequência enviada.
- **Controle de fluxo**: Os mecanismos integrados de controle de fluxo (janela deslizante) e congestionamento do TCP evitam que um remetente rápido sobrecarregue um receptor lento ou a própria rede.
- **Deteção de erros**: Checksums automáticos garantem integridade.

Alternativa Rejeitada - UDP: UDP oferece menor latência mas requer implementação manual de mecanismos de confiabilidade (ACK, retransmissão, ordenação). Para aplicação de chat, onde confiabilidade

supera performance extrema, TCP é melhor.

Overhead: O custo dessa confiabilidade é um maior overhead (cabeçalhos de 20 bytes) e maior latência inicial (3-way handshake) em comparação com o UDP. No entanto, para esta aplicação, a consistência da comunicação é um requisito mais crítico do que a latência de microssegundos. Adotar UDP exigiria reimplementar uma camada de confiabilidade, o que seria equivalente a "reinventar a roda" de forma menos eficiente.

3.2 Modelo de Concorrência

A concorrência no servidor é gerenciada através de um modelo Thread-por-Cliente.

Arquitetura Thread-per-Client:

```
client_thread = threading.Thread( # É alocada uma thread para cada 'client'
    target=self.handle_client,
    args=(client_socket, client_address)
)
# As threads são iniciadas
client_thread.daemon = True
client_thread.start()
```

Análise de Alternativas:

Modelo	Vantagens	Desvantagens	Decisão
Thread-per-Client	Simplicidade de código, isolamento de falhas	Limitação de escala, consumo de memória por thread	Escolhido
Thread Pool	Controle de recursos, reutilização de threads	Maior complexidade de gerenciamento de tarefas	Rejeitado
Async/Event-Loop	Alta escalabilidade (milhares de conexões)	Complexidade de código (async/await), debugging	Rejeitado

Justificativas: Para ambiente acadêmico com <50 usuários simultâneos, simplicidade de implementação e debugging supera limitações de escalabilidade.

3.3 Sincronização de Dados Compartilhados

Estratégia de Locking:

```
self.client_lock = threading.Lock() # Protege lista de clientes
self.group_lock = threading.Lock() # Protege estrutura de grupos

# Uso típico
with self.client_lock:
    if username in self.clients:
        del self.clients[username]
```

Justificativas:

- 1. **Granularidade Fina:** Locks separados para clientes e grupos minimizam contenção
- 2. **Duração Mínima:** Locks mantidos apenas durante operações atômicas
- 3. **Ordem Consistente:** Sempre adquirir client_lock antes de group_lock para evitar deadlocks
- 4. **Copy-on-Read:** Criar cópias de estruturas antes de iterar fora do lock

Alternativa Rejeitada - Lock Global: Um único lock para todas as estruturas simplificaria código mas criaria gargalo de performance.

3.4 Serialização e Transferência de Dados

Utilizamos JSON para serialização de mensagens e Base64 para encapsular arquivos. Adotamos framing por comprimento: cada mensagem é enviada como [4 bytes (tamanho)] + [payload JSON em UTF-8]. Todos os envios usam sendall.

Escolha do JSON:

(framing + leitura exata + sendall)

```
import json, struct

def send_json(sock, obj):
    data = json.dumps(obj).encode('utf-8')
    header = struct.pack('!I', len(data))    # 4 bytes com o tamanho
    sock.sendall(header)                    # garante envio completo
    sock.sendall(data)

def _recv_exact(sock, n):
    buf = bytearray()
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("Conexão fechada pelo par")
        buf.extend(chunk)
    return bytes(buf)

def recv_json(sock):
    header = _recv_exact(sock, 4)            # lê exatamente 4 bytes
    (length,) = struct.unpack('!I', header)
    payload = _recv_exact(sock, length)     # lê exatamente 'length' bytes
    return json.loads(payload.decode('utf-8'))
```

Análise Técnica:

Formato	Tamanho	Performance	Legibilidade	Interoperabilidade
JSON	100%	Baseline	Alta	Universal

Formato	Tamanho	Performance	Legibilidade	Interoperabilidade
Protocol Buffers	~40%	3-5x mais rápido	Baixa	Limitada
Pickle	~60%	2x mais rápido	Nula	Python apenas

Codificação Base64:

```
# Codificação no cliente
with open(file_path, 'rb') as f:
    file_data = base64.b64encode(f.read()).decode('utf-8')

# Decodificação no servidor/destinatário
with open(output_path, 'wb') as f:
    f.write(base64.b64decode(file_data))
```

Alternativa Rejeitada - Transferência Binária: Separar canal de dados binários adicionaria complexidade significativa ao protocolo.

Justificativas:

- **JSON:** Oferece o melhor equilíbrio entre legibilidade humana (facilitando o debugging), suporte universal e flexibilidade. O overhead de desempenho em comparação com formatos binários como Protocol Buffers é um trade-off aceitável pela simplicidade de desenvolvimento.
- **Base64:** Esta codificação permite a unificação do canal de comunicação. Ao converter dados binários em texto ASCII seguro, os arquivos podem ser enviados dentro da mesma estrutura JSON das mensagens de texto, utilizando a mesma conexão TCP. O aumento no tamanho dos dados é um custo aceitável por essa simplicidade arquitetural.

4. Implementação de Funcionalidades Críticas

4.1 Controle de Acesso a Grupos

A segurança e a privacidade dos grupos são garantidas por validações rigorosas no servidor. Toda ação relacionada a um grupo (enviar mensagem, adicionar membro, listar membros) verifica primeiro se o solicitante pertence àquele grupo, negando a operação caso contrário.

Modelo de Membership:

```
def handle_group_message(self, message: dict) -> dict:
    # Validação rigorosa de membership
    if sender not in self.groups[group_name]:
        return {
            'status': 'error',
            'message': 'Você não é membro do grupo. Peça para alguém te
adicionar.'
        }
```

Design de Segurança:

- Apenas membros podem enviar mensagens/arquivos
- Qualquer membro pode adicionar outros usuários
- Lista de membros visível apenas para membros
- Sem conceito de administrador (simplicidade)

4.2 Distribuição de Mensagens em Grupo

Algoritmo de Broadcast:

```
# Obtém membros atomicamente
with self.group_lock:
    group_members = self.groups[group_name].copy()

# Distribui fora do lock para evitar contenção
delivered_count = 0
with self.client_lock:
    for member in group_members: # Itera sobre cada membro do grupo
        if member != sender and member in self.clients: # Se for um membro e não
            quem envia a mensagem
            try:
                # Envia a mensagem para cada membro
                member_socket = self.clients[member]
                member_socket.send(notification_json)
                delivered_count += 1
            except:
                continue # Ignora falhas individuais
```

Características:

- Best-effort delivery para membros conectados
- Falhas individuais não afetam outros membros
- Contador de entrega para feedback ao remetente ("Mensagem enviada para x membros")

4.3 Tratamento de Falhas

Estratégias Implementadas:

1. Desconexão Abrupta:

Caso o membro seja desconectado do sistema por qualquer motivo, sem que se desconecte por vontade própria, selecionando a opção de "Sair", então uma mensagem de desconexão é enviada no Servidor:

```
except ConnectionResetError:
    print(f"Cliente {client_address} desconectou abruptamente")
finally:
    if username:
```

```
with self.client_lock:
    del self.clients[username]
```

2. Mensagens Malformadas:

Caso a mensagem enviada apresente algum erro ao ser decodificada na recepção, então o aviso de erro enviado é o seguinte:

```
except json.JSONDecodeError:
    error_response = {
        'type': 'error',
        'message': 'Formato de mensagem inválido'
    }
```

3. Thread Isolation:

Cada cliente em thread separada impede que falha individual afete outros usuários.

5. Testes e Validação

Para garantir a corretude funcional e a robustez do sistema, foi executado um roteiro de testes abrangente, simulando um ambiente com 5 usuários conectados simultaneamente (Alice, Bob, Carlos, Diana, Eduardo). Os testes foram conduzidos em ambiente de rede local (localhost) para isolar variáveis de rede.

5.1 Ambiente de Teste

Configuração:

- Sistema: Ubuntu 22.04 LTS, Python 3.10.12
- Hardware: Intel i5-9600K, 32GB RAM
- Rede: Localhost (eliminação de variáveis de rede)

5.2. Metodologia de Teste

A validação foi realizada através de testes de sistema manuais, cobrindo todas as funcionalidades especificadas. Os resultados foram verificados em tempo real nos terminais dos clientes e através dos logs do servidor, além da inspeção dos arquivos transferidos no sistema de arquivos.

5.3. Cenários de Teste e Resultados Consolidados

A tabela abaixo resume os principais cenários testados e seus resultados, confirmando que 100% das funcionalidades implementadas operaram conforme o esperado.

Categoria	Cenário de Teste	Resultado Esperado	Status
Conexão e Sessão	Conexão de 5 clientes com usernames únicos.	Todos se conectam com sucesso. O servidor registra as 5 sessões ativas.	☒ Passou

Categoria	Cenário de Teste	Resultado Esperado	Status
	Listagem de usuários online (<code>/list_users</code>).	Cliente solicitante recebe a lista completa dos 5 usuários conectados.	☒ Passou
Mensagens Privadas	Envio de mensagem de Alice para Bob.	Bob recebe a mensagem instantaneamente. Outros clientes não a recebem.	☒ Passou
Gerenciamento de Grupos	Alice cria o grupo "Trabalho"; Diana cria o grupo "Amigos".	Grupos são criados com sucesso. Alice e Diana se tornam os primeiros membros.	☒ Passou
	Alice adiciona Bob e Carlos ao grupo "Trabalho".	Bob e Carlos recebem notificação de que foram adicionados.	☒ Passou
	Bob verifica a lista de membros do grupo "Trabalho".	Bob recebe a lista correta contendo Alice, Bob e Carlos.	☒ Passou
Mensagens em Grupo	Alice envia uma mensagem para o grupo "Trabalho".	A mensagem é entregue a Bob e Carlos. A própria Alice não a recebe de volta.	☒ Passou
Controle de Acesso	Eduardo (não-membro) tenta enviar mensagem para o grupo "Trabalho".	Servidor rejeita a mensagem com um erro de permissão.	☒ Passou
Transferência de Arquivos	Alice envia um arquivo (<code>.txt</code>) privado para Bob.	Bob recebe o arquivo, que é salvo em seu diretório <code>client_downloads</code> .	☒ Passou
	Carlos envia um arquivo (<code>.pdf</code>) para o grupo "Trabalho".	Alice, Bob e Eduardo recebem o arquivo em seus respectivos diretórios.	☒ Passou
	Verificação de integridade dos arquivos recebidos.	O hash MD5 dos arquivos recebidos é idêntico ao dos arquivos originais.	☒ Passou
Robustez	Desconexão abrupta de um cliente (Ctrl+C).	Servidor detecta a desconexão, remove o usuário da lista de ativos e os demais clientes continuam operando normalmente.	☒ Passou

5.4. Análise dos Resultados

Os testes demonstraram que a arquitetura multithreaded do servidor foi capaz de gerenciar 5 conexões simultâneas de forma eficiente e sem contenção perceptível. Os mecanismos de sincronização (`Locks`) se provaram eficazes em prevenir condições de corrida durante operações concorrentes, como adição de membros e envio de mensagens. O protocolo de comunicação baseado em JSON foi robusto o suficiente para todas as operações, incluindo a transferência de arquivos via Base64, que ocorreu sem corrupção de dados. O controle de acesso a grupos funcionou como especificado, garantindo a privacidade das conversas.

Obs: Obteve 100% de integridade para arquivos até 10MB testados.

6. Limitações e Melhorias

6.1 Limitações Arquiteturais

Escalabilidade:

- Modelo thread-per-client limita a uma baixa quantidade de usuários simultâneos
- Servidor centralizado cria ponto único de falha
- Estruturas de dados em memória não persistem restarts

Performance:

- Codificação Base64 adiciona overhead de 33%
- JSON parsing mais lento que formatos binários
- Sem otimizações para arquivos grandes

6.2 Melhorias Propostas

Curto Prazo:

- Implementação de database para persistência
- Compressão de arquivos antes de codificação
- Autenticação com hash de senhas

Médio Prazo:

- Migração para asyncio:

```
# Migração para asyncio
async def handle_client(self, websocket, path):
    # Suporta milhares de conexões concorrentes
```

Longo Prazo:

- Arquitetura distribuída com microserviços
- Load balancing entre múltiplos servidores
- Criptografia end-to-end

7. Conclusão

O sistema implementado atende completamente aos requisitos especificados, demonstrando aplicação prática de conceitos fundamentais de sistemas distribuídos. As decisões técnicas - uso de TCP para confiabilidade, threading para concorrência, JSON para simplicidade.

A arquitetura cliente-servidor centralizada, embora apresente limitações de escalabilidade, oferece simplicidade de implementação e debugging compatível com objetivos educacionais. Os testes validaram robustez do sistema com 100% de sucesso em cenários de múltiplos usuários.

Para evoluções futuras temos algumas opções, seja através de otimizações de performance, migração para arquitetura assíncrona, ou implementação de funcionalidades avançadas como persistência e criptografia.